

«Talento Tech»

React JS

Clase 06



Clase N° 6 | Formularios y carga de imágenes

Índice

Formularios y carga de imágenes

Otros eventos:

Evento onChange

Evento onSubmit

Creando nuestro formulario.

Paso 1: Un formulario presentacional simple

Paso 2. Creamos el componente contenedor del Formulario

Paso 3: Integramos en App.jsx

Subiendo imágenes a Imgbb

Modificando nuestro contenedor - FormularioContainer.jsx,

Objetivos de la Clase:

- Construir un **formulario presentacional** simple y reutilizable.
- Manejar los eventos onChange y onSubmit para crear un **componente controlado**.
- Aplicar el patrón **Contenedor/Presentacional** para manejar la lógica del formulario.
- Integrar la **subida de imágenes** a un servicio externo (Imgbb) y mostrar el resultado en tiempo real.

Otros eventos:

La clase 4 vimos el evento onClick que nos permitía accionar frente a al clic del usuario en algún elemento: en nuestro caso en un boton. Con las mismas reglas que usamos onClick, vamos a recordar estos dos nuevos eventos que nos van a permitir interaccionar con el formulario: onChange y onSubmit.

Evento onChange

Captura cambios en tiempo real en elementos de formulario (inputs, selects, textareas).

Por ejemplo:

```
<input
  value={nombre}
  onChange={(e) => setNombre(e.target.value)}
/>
```

Descripción:

1. Usuario escribe → Dispara evento onChange
2. e.target.value → Obtiene el valor actual del input
3. setNombre() → Actualiza el estado de React
4. Re-render → El input muestra el nuevo valor

Características:

- ✓ Se ejecuta con cada tecla presionada
- ✓ Sincroniza el estado con lo que el usuario ve
- ✓ Esencial para formularios controlados

Evento onSubmit

Maneja el envío del formulario cuando el usuario hace submit.

Por ejemplo:

```
<form onSubmit={(e) => {
  e.preventDefault();
  console.log('Formulario enviado!');}}>
```

1. Usuario envía form (Enter o click en botón)
2. e.preventDefault() → Evita recargar toda la página
3. Ejecuta lógica → Validación, enviar datos, etc. (console.log en nuestro caso)

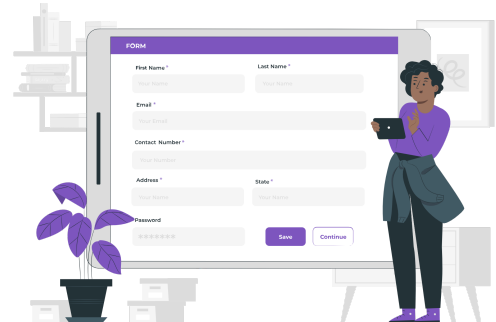
Características:

- ✓ Se ejecuta una vez al enviar el form
- ✓ Previene la recarga de la página, comportamiento que viene por default.
- ✓ Ideal para procesar datos finales

Creando nuestro formulario.

Vamos a usar estos eventos en un formulario real, que nos permita capturar la información y guardarla. Para eso dividimos la creación del formulario en 3 pasos:

1. Creamos el formulario
2. Lo hacemos parte de nuestro patron Contenedor/Presentacional
3. Lo integramos a nuestra aplicación



Paso 1: Un formulario presentacional simple

Recordemos lo que vimos en la clase 4: los componentes presentacionales solo se preocupan por **cómo se ven las cosas**. No tienen lógica compleja. Vamos a empezar creando un formulario simple que solo renderiza los campos que necesitamos.

Crear el componente FormularioProductos.jsx

1. Dentro de tu carpeta /componentes, creá una subcarpeta llamada FormularioProductos y su componente.
2. Este componente solo va a renderizar la estructura HTML del formulario.
3. Con algunos detalles de estilos que le podamos dar, el código nos quedará de la siguiente manera:

```
// En src/componentens/FormularioProducto/FormularioProducto
import React from 'react';

// Por ahora, es un componente súper simple. Solo muestra el HTML.
export function FormularioProducto() {
  const formStyle = {
    display: 'flex',
    flexDirection: 'column',
    maxWidth: '24rem',
    margin: '3rem auto',
    padding: '1.5rem',
    border: '1px solid #ddd',
    borderRadius: '8px',
    gap: '16px'
  };

  return (
    <form style={formStyle}>
      <h3>Agregar Nuevo Producto</h3>
      <div>
        <label>Nombre del Producto:</label>
```

```

    <input type="text" placeholder="Ej: Teclado Mecánico" />
  </div>
  <div>
    <label>Precio:</label>
    <input type="number" placeholder="Ej: 95" />
  </div>
  <div>
    <label>Stock:</label>
    <input type="number" placeholder="Ej: 5" />
  </div>
  <div>
    <label>Imagen:</label>
    <input type="file" placeholder="https://..." />
  </div>
  <button type="submit">Guardar Producto</button>
</form>
);
}

```

Si renderizás este componente en tu App.jsx, verás un formulario. Lindo, pero inútil. Si le das al botón, la página se recarga.

Dándole agilidad

Modifiquemos nuestro componente para que reciba 3 props: **datosForm**, **manejarCambio** y **manejarEnvio**. Estos nos van a servir para:

- **datosForm**: Un objeto que contiene los valores actuales de todos los campos del formulario (nombre, precio, stock, etc.). Lo usaremos para poblar el atributo value de cada input.
- **manejarCambio**: Una función que se ejecutará cada vez que el usuario escriba en un input. La conectaremos al evento onChange.
- **manejarEnvio**: Una función que se ejecutará cuando el usuario envíe el formulario. La conectaremos al evento onSubmit de la etiqueta <form>.

Al formulario le agregamos la opción de onSubmit con su prop correspondiente. También modificamos los input para que podamos manejar los datos con sus props.

```

// En src/componentes/FormularioProducto/FormularioProducto.jsx
import React from 'react';
// Recibe los datos y las funciones por props.
export function FormularioProducto({ datosForm, manejarCambio, manejarEnvio })
{
  const formStyle = { // mismo diseño };

  return (
    <form style={formStyle} onSubmit={manejarEnvio}>
      <h3>Agregar Nuevo Producto</h3>
      <div>

```

```

    <label>Nombre del Producto:</label>
    <input
      type="text"
      placeholder="Ej: Teclado Mecánico"
      name="nombre" // Atributo clave para identificar el input
      value={datosForm.nombre}
      onChange={manejarCambio}
    />
  </div>
  <div>
    <label>Precio: $</label>
    <input
      type="number"
      placeholder="Ej: 95"
      name="precio" // Atributo clave
      value={datosForm.precio}
      onChange={manejarCambio}
    />
  </div>
  // Continuamos con los otros 2 atributos del formulario.
  <button type="submit">Guardar Producto</button>
</form>
);
}

```

Paso 2. Creamos el componente contenedor del Formulario

Una vez que tenemos nuestro componente presentacional listo, vamos a generar su contenedor. Para eso creamos un nuevo componente llamado FormularioContainer, donde importamos useState y el componente FormularioProducto.

Este componente será el cerebro de la operación.

1. Manejo del Estado con useState

Lo primero que hacemos es utilizar useState para crear una variable de estado llamada datosForm, donde agrupamos todos los campos en un único objeto(nombre, precio, etc.),

2. Lógica de Actualización: manejarCambio

Creamos una única función, manejarCambio, que permite actualizar cualquier campo del formulario, haciendo que cada vez que el usuario escribe, se dispare el evento onChange.

Recibe el objeto evento y extrae name (el atributo name que le dimos a cada input, como "nombre" o "precio") y value (el texto que el usuario está escribiendo).

```
const manejarCambio = (evento) => {
  const { name, value } = evento.target;
  setDatosForm({
    ...datosForm, [name]: value
  });
};
```

3. Lógica de Envío: manejarEnvio

Esta función se ejecuta cuando el usuario presiona el botón "Guardar producto". Primero llama a `evento.preventDefault()` para evitar el comportamiento por defecto de recargar la página entera al enviarse.

Después centralizamos todo lo que debe ocurrir con los datos ya completos. Por ahora, simplemente los mostramos en la consola, pero próximamente desde aquí haríamos la llamada a una API para guardar el producto en una base de datos.

```
const manejarEnvio = (evento) => {
  evento.preventDefault();
  console.log('Enviando los siguientes datos a la API:', datosForm);
};
```

4. Return

Este no renderiza HTML directamente. Su única función es renderizar al componente presentacional `FormularioProducto` y pasarle, a través de props, todo lo que necesita para funcionar.

```
// En src/contenedores/FormularioContainer/FormularioContainer.jsx
import React, { useState } from 'react';
import { FormularioProducto } from '../FormularioProducto/FormularioProducto';
export function FormularioContainer() {
  const [datosForm, setDatosForm] = useState({
    nombre: '',
    precio: '',
    stock: '',
    urlImagen: ''
  });
  const manejarCambio = (evento) => {
    const { name, value } = evento.target;
    setDatosForm({
      ...datosForm,
      [name]: value
    });
  };
};
```

```
const manejarEnvio = (evento) => {
  evento.preventDefault();
  console.log('Enviando los siguientes datos a la API:', datosForm);
};
return (
  <FormularioProducto
    datosForm={datosForm}
    manejarCambio={manejarCambio}
    manejarEnvio={manejarEnvio}
  />
);
}
```

En próximas clases enviaremos estos datos a nuestro servidor backend.

Paso 3: Integramos en App.jsx

Una vez tenemos nuestro formulario armado, en app.jsx vamos a importarlo y lo agregamos donde queremos.

```
// En src/App.jsx
import {FormularioContainer} from
'./componentes/FormularioContainer/FormularioContainer'
function App() {
  return (
    <Layout>
      {/* Todo lo que pongamos acá adentro irá donde estaba {children} */}
      <h1>¡Bienvenidos a mi página!</h1>
      <p>Este es el contenido principal.</p>
      <ItemListContainer Mensaje="Nuestros productos destacados" />
      <Productos Mensaje="Productos locales"></Productos>
      <FormularioContainer/>
    </Layout>
  );
}
```

Si ejecutamos la aplicación, desde nuestra terminal, con npm run dev, veremos el formulario simple que nos permite “cargar” datos...El próximo paso es cargar imágenes.

Subiendo imágenes a Imgbb

Imgbb es un servicio gratuito de alojamiento de imágenes en línea que te permite subir imágenes a sus servidores y te proporciona un enlace directo para compartir esa imagen en foros, redes sociales, blogs o cualquier sitio web.



Características:

- **Enlaces directos:** Te da URLs (como `i.ibb.co/.../imagen.jpg`) para insertar las imágenes donde quieras.
- **Si no te registraste:** Podés subir imágenes sin necesidad de crear una cuenta, pero se borran después de un tiempo.
- **Álbumes y gestión:** Si creas una cuenta gratuita, puedes organizar tus imágenes en álbumes y gestionarlos.

Primero, necesitamos una **API Key** de Imgbb. Es un código único que nos identifica y nos autoriza a usar su servicio.

1. **Regístrate** en imgbb.com. Es gratis y podés ingresar con tu cuenta de gmail.
2. Una vez dentro, andá a la sección **"Acerca"** (Arriba a la izquierda) y luego a **"API"**.
3. Hacé clic en **"GET API-KEY"** y te mostrará tu clave de API. Copiala y guardala que la vamos a necesitar en un momento.
4. Lo ideal es guardar la API-KEY en una variable en un archivo **.env** en la raíz de tu proyecto y luego importarla para su uso en los componentes correspondientes. Por el momento podés usarlo como te quede más cómodo.

¡Es importante!

Ahora, modifiquemos nuestro formulario para que acepte archivos, porque como el input de la imagen se manejará de forma un poco diferente, le pasaremos una nueva prop llamada **manejarCambiarImagen**.

Recordá agregarla en la función `FormularioProducto` y modificarla en el input de tipo `file`.

Modificando nuestro contenedor - FormularioContainer.jsx,

Vamos a implementar la lógica para tomar el archivo que el usuario seleccionó, enviarlo a Imgbb y, una vez que tengamos la URL, simular el envío final de todos los datos juntos.

Modificaremos el archivo en 3 pasos:

1. Modificamos el estado para guardar el archivo imagen.
2. Creamos la función manejarCambioImagen.
3. Modificar manejarEnvio.

1. Crear un nuevo estado para guardar el archivo de imagen seleccionado:

Para eso, lo eliminamos del useState que teníamos y lo pasamos aparte.

```
// En src/contenedores/FormularioContainer/FormularioContainer.jsx
const [datosForm, setDatosForm] = useState({
  nombre: '',
  precio: '',
  stock: ''
  // Quitamos la urlImagen de aca porque la obtendremos después de la subida
});

// 1. Nuevo estado para el archivo de imagen
const [imagenFile, setImagenFile] = useState(null);
```

2. Implementar la función manejarCambioImagen para actualizar ese estado:

```
const manejarCambio = (evento) => {
  // Se mantiene igual
};

// 2. Nueva función para manejar el cambio del input de tipo "file"
const manejarCambioImagen = (evento) => {
  setImagenFile(evento.target.files[0]);
};
```

evento.target.files es un array con los archivos seleccionados, pero como solo permitimos uno, tomamos el primero (índice 0).

3. Modificar manejarEnvio para que:

- Primero, suba la imagen a Imgbb usando fetch.
- Segundo, con la URL de la imagen ya obtenida, construya el objeto final del producto.
- Finalmente, muestre en consola el objeto completo, listo para ser enviado a nuestro propio backend en el futuro.

Para enviar un archivo necesitamos usar un objeto especial llamado FormData, que está diseñado para enviar archivos y datos de formulario.

```
const manejarEnvio = async (evento) => {
  evento.preventDefault();
  // Validamos que el usuario haya seleccionado una imagen
  if (!imagenFile) {
    alert("Por favor, selecciona una imagen para el producto.");
    return;
  }
  // --- Lógica para subir la imagen a Imgbb ---
  const apiKey = 'TU-API-KEY'; // 🚨 ¡Reemplazá esto con tu clave!
  const formData = new FormData();
  formData.append('image', imagenFile);

  try {
    console.log("Subiendo imagen a Imgbb...");
    const respuestaImgbb = await
fetch(`https://api.imgbb.com/1/upload?key=${apiKey}`, {
      method: 'POST',
      body: formData,
    });

    const datosImgbb = await respuestaImgbb.json();

    if (datosImgbb.success) {
      console.log("Imagen subida con éxito. URL:", datosImgbb.data.url);

      // Unimos la URL de la imagen con el resto de los datos del
formulario
      const productoCompleto = {
        ...datosForm,
        // Agregamos la URL obtenida
        urlImagen: datosImgbb.data.url
      };
    }
  }
}
```

```

// Por el momento hacemos un console.log
    console.log('Enviando los siguientes datos COMPLETOS a la API:',
productoCompleto);

    } else {
        throw new Error('La subida de la imagen a Imgbb falló.');
```

Una vez que enviamos el formulario, veremos por consola los datos cargados. También si actualizamos nuestra cuenta en Imgbb, veremos que tenemos las imágenes cargadas correctamente.

Reflexión final

¡Felicitaciones! En esta clase, dominaste los formularios. Al aprender a gestionar los eventos onChange y onSubmit, y al aplicar el patrón Contenedor/Presentacional, pasaste de crear interfaces estáticas a construir componentes controlados, dinámicos y reutilizables. Sentamos las bases para capturar y manejar la información del usuario, una competencia esencial para cualquier desarrollador. Este conocimiento es la puerta de entrada a la creación de experiencias de usuario ricas y funcionales. ¡Siga adelante!

TalentoLab - Proyecto final



Descripción de tu tarea:



En este ejercicio, vas a mejorar la experiencia de usuario de tu aplicación eCommerce integrando un indicador de carga visual mientras se sube la imagen del producto. Esto previene que el usuario envíe el formulario varias veces y le informa que el proceso está en curso.

Mejorar la experiencia de usuario con un estado de carga

- **Añadir y gestionar el estado de carga en NewProductContainer.jsx**
 - **Crear el estado loading:** Justo debajo de tus otros estados, agregá uno nuevo para controlar el estado de carga. Inicialízalo en false.
 - **Activar el estado de carga:** Dentro de la función handleFormSubmit, la primera acción que debes realizar es cambiar el estado loading a true.
 - **Desactivar el estado de carga:** Es crucial desactivar el estado de carga (setLoading(false)) cuando el proceso finaliza, ya sea con éxito o con un error. Asegurate de añadirlo al final del bloque try y también en el bloque catch. Usar finally es una excelente práctica para este caso, ya que el código dentro de este bloque se ejecutará siempre, sin importar si el try tuvo éxito o si se produjo un catch.
 - **Pasar el estado como prop:** Envía el estado loading al componente ProductForm a través de sus props.
- **Utilizar el estado de carga en ProductForm.jsx**
 - **Recibir la prop:** Desestructura la prop loading que ahora recibe el componente.
 - **Modificar el botón de envío:** Usa la prop loading para cambiar dinámicamente el texto y el estado del botón. Puedes hacerlo con un operador ternario. El botón debe estar deshabilitado (disabled) cuando loading sea true.

Materiales y Recursos Adicionales:

- ★ [Eventos en React](#)
 - ★ [ImgBB Documentación](#)
 - ★ [Patrones de diseño de React: El patrón contenedor/presentativo](#)
-

Preguntas para Reflexionar:

- ¿Qué ventajas concretas nos ofrece el patrón de componentes controlados a dejar que el DOM maneje el estado de los inputs?
 - ¿Qué consideraciones de seguridad y experiencia de usuario deberíamos tener en cuenta al permitir la subida de archivos de imágenes desde el cliente a un servicio externo?
 - Si tuvieras que construir un formulario mucho más complejo (con validaciones, múltiples pasos y campos dependientes), ¿cómo evolucionaría la implementación del patrón Contenedor/Presentacional que vimos en clase?
-

Próximos Pasos:

- Instalar y configurar la librería react-router-dom en nuestro proyecto.
- Comprender el funcionamiento del ruteo en una Aplicación de Página Única (SPA).
- Implementar un sistema de navegación principal utilizando rutas estáticas y el componente Link.



Buenos Aires
aprende
Agencia de Políticas para el Futuro

BA Buenos
Aires
Ciudad